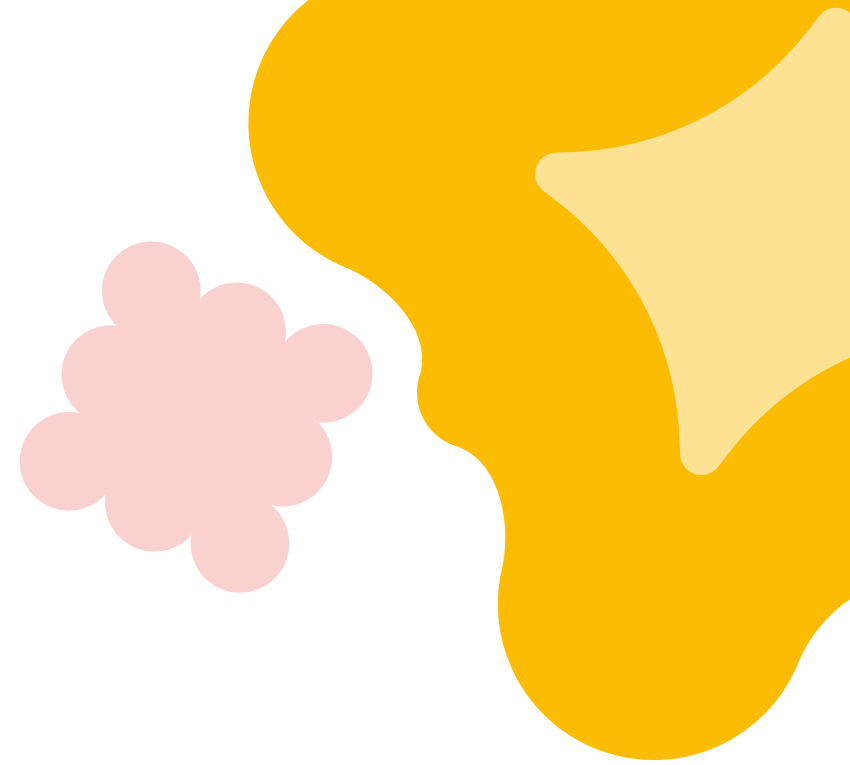




Hands-on example

Building a configurable document processor

What you'll build: A document processor that respects user preferences (from `user:` namespace) and tracks processing status.

Step 1: Create the project

Shell

```
adk create doc_processor
cd doc_processor
```

Step 2: Write the agent

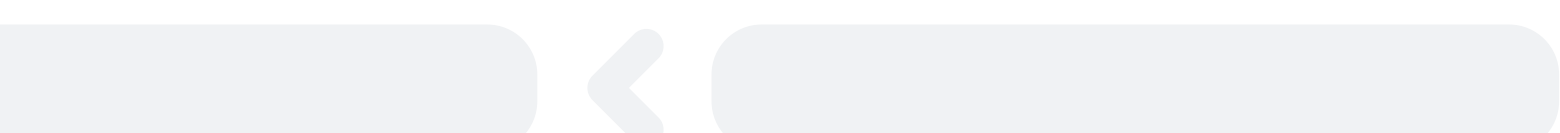
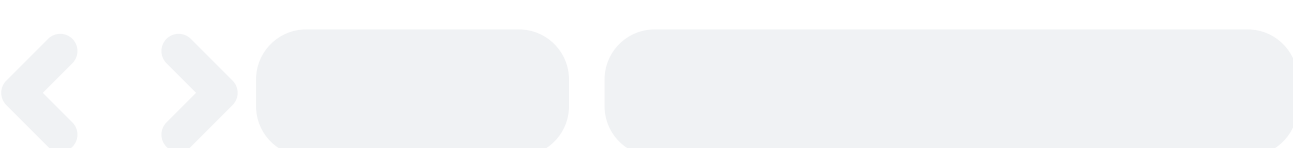
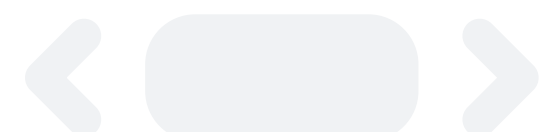
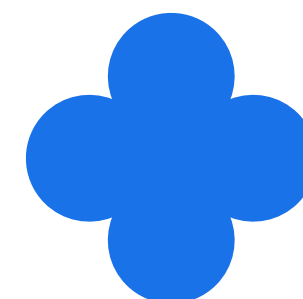
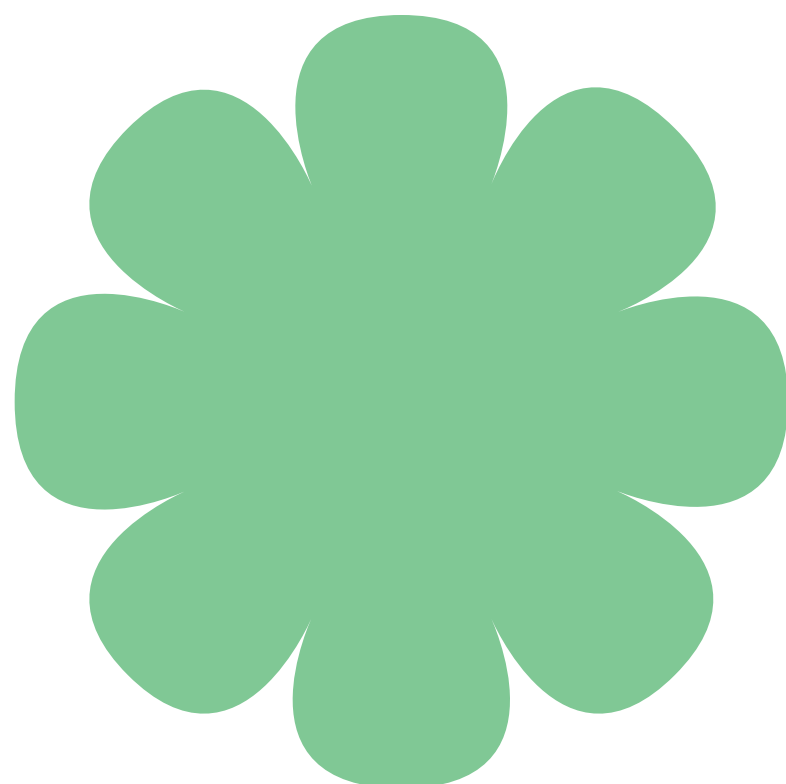
Replace `agent.py` with:

Python

```
"""
```

```
Configurable Document Processor
Demonstrates state namespaces in multi-agent workflows.
```

```
Reference: https://google.github.io/adk-docs/state
"""
```



```

from google.adk.agents import LlmAgent, SequentialAgent, ParallelAgent

# =====
# STAGE 1: Intake - Extract document content
# =====
intake_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='intake',
    output_key='document_content', # Session namespace
    instruction=''
    Extract and structure the document content from the user's input.

    Return a clean, structured version of the content.
    ''
)

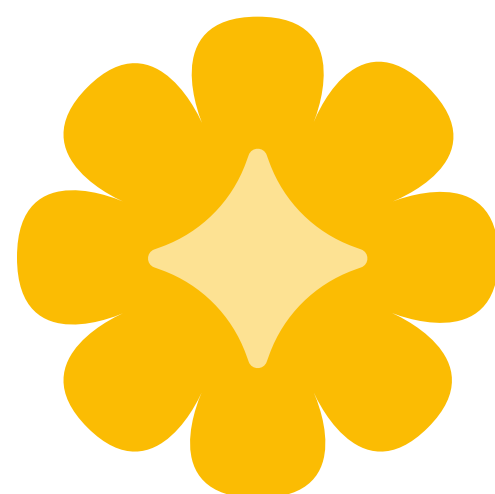
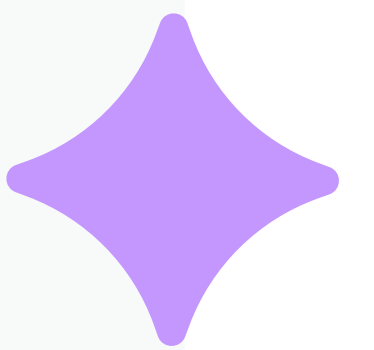
# =====
# STAGE 2: Parallel Analysis - Multiple analyzers running together
# =====
topic_analyzer = LlmAgent(
    model='gemini-2.5-flash',
    name='topic_analyzer',
    output_key='topics', # Distinct key for parallel
    instruction=''
    Analyze the document for main topics:
    {document_content}

    Return the top 3-5 topics.
    ''
)

sentiment_analyzer = LlmAgent(
    model='gemini-2.5-flash',
    name='sentiment_analyzer',
    output_key='sentiment', # Distinct key for parallel
    instruction=''
    Analyze the document sentiment:
    {document_content}

    Return: positive, negative, or neutral with explanation.
    ''
)

```




```

parallel_analysis = ParallelAgent(
    name='parallel_analysis',
    sub_agents=[topic_analyzer, sentiment_analyzer]
)

# =====
# STAGE 3: Summary - Respects user preferences
# =====
summarizer = LlmAgent(
    model='gemini-2.5-flash',
    name='summarizer',
    output_key='summary', # Session namespace
    instruction=''
    Create a document summary.

    Document content: {document_content}
    Topics identified: {topics}
    Sentiment: {sentiment}

    User preference for format: {user:summary_format}
    (If no preference, use "bullet points")

    Create summary matching the user's preferred format.
    ...
)

# =====
# COMPLETE WORKFLOW
# =====
root_agent = SequentialAgent(
    name='document_processor',
    sub_agents=[
        intake_agent,      # Stage 1: Extract
        parallel_analysis,  # Stage 2: Analyze (parallel)
        summarizer          # Stage 3: Summarize
    ]
)

```



What this demonstrates:

- Session namespace: `document_content`, `topics`, `sentiment`, `summary` persist in conversation
- User namespace: `{user:summary_format}` reads user preference
- Parallel coordination: Distinct keys for parallel analyzers
- State flow: Each stage builds on previous outputs

Step 3: Run and test

```
Shell
adk web
```

Navigate to <http://localhost:8000>

Step 4: Test state flow

Test 1: Process a document

You: `Process this document: "The new product launch exceeded expectations. Sales jumped 40% in the first week. Customer feedback has been overwhelmingly positive, with 95% satisfaction ratings."`

Expected behavior:

1. Intake extracts and structures content
2. Topic and sentiment analyzers run in parallel
3. Summarizer creates summary (default format since no user preference)
4. All state is preserved for follow-up questions

What to notice:

- State flows through the pipeline
- Parallel agents don't conflict (distinct keys)
- Summarizer uses default since `user:summary_format` isn't set



Test 2: Follow-up in same conversation

You: `What topics did you identify?`

Expected: The agent can reference `topics` from state since it persists in the conversation.

Key takeaways

Effective state management is the foundation of complex multi-agent workflows. By understanding how state flows between agents and leveraging Lesson 4’s namespace patterns, you can build workflows that maintain context, respect user preferences, and coordinate parallel execution seamlessly.

State flow basics

- All agents in a workflow share one session state
- `output_key` writes to state, `{key}` reads from state
- State accumulates as workflow progresses

Namespace usage in workflows

Namespace	Workflow Use
Session	Workflow outputs and intermediate results
<code>user:</code>	User preferences that affect processing
<code>app:</code>	Global configuration for workflows
<code>temp:</code>	Rarely needed in basic workflows

Parallel state coordination

- Use distinct `output_key` for each parallel agent
- Aggregator reads all keys using `{key}` templating
- State is merged after parallel stage completes

